

Software Requirements Specification (SRS)

Project: Quantum Bluff

Version 1.0 - Milestone M1 Validation

Project Leader: Omar

Tech Lead: Mohamed

QA/Analyst: Linda

Scientific Advisor: Cyrille Baudrier

February 15, 2026

Contents

0.1	Introduction	2
0.1.1	Purpose	2
0.1.2	Project Scope	2
0.1.3	Definitions and Roles	2
0.2	Overall Description	2
0.2.1	Product Perspective	2
0.2.2	Operational Environment	2
0.3	Functional Requirements	2
0.3.1	Core Game Mechanics	2
0.3.2	Data Persistence	3
0.3.3	Required Activity Diagram	4
0.4	Brainstorming: Speculative "Hidden Bets" Mechanics	6
0.5	Finalizing Quantum Bluff Game Theory	7
0.6	Technical Specifications	7
0.6.1	Technology Stack	7
0.6.2	Architecture Requirements	7
0.7	Validation and Quality Assurance	8
0.7.1	Validation Protocol (M1)	8
0.7.2	Non-Functional Requirements	8

0.1 Introduction

0.1.1 Purpose

The purpose of this document is to define the functional and technical specifications for the **Quantum Bluff** desktop application. This document serves as the primary reference for development, quality assurance, and project validation.

0.1.2 Project Scope

Quantum Bluff is a high-stakes strategy game involving complex probability mechanics (*Quantum Speculation*). The software is developed as a ****100% local desktop client****, prioritizing mathematical integrity and real-time performance without the need for an internet connection.

0.1.3 Definitions and Roles

- **PL (Omar):** Global oversight and strategic validation.
- **Tech Lead (Mohamed):** Technical architecture and code integrity.
- **QA (Linda):** Validation of requirements and regression testing.
- **Scientific Lead (C. Baudrier):** Modeling of the Hidden Bets algorithms.

0.2 Overall Description

0.2.1 Product Perspective

The application is a standalone Java-based software using JavaFX for the user interface. It operates entirely on the user's machine, ensuring data privacy and offline accessibility.

0.2.2 Operational Environment

- **Operating System:** Windows 10/11, macOS, Linux.
- **Runtime:** Java Runtime Environment (JRE) 17 or higher.
- **Connectivity:** None (100% Offline).

0.3 Functional Requirements

0.3.1 Core Game Mechanics

FR-1 Local Session Management: The system shall allow users to start, pause, and save game sessions locally without account creation.

FR-2 Hidden Bets Engine: The system shall implement the "Quantum Speculation" mechanic where players bet on hidden probabilistic outcomes.

FR-3 Odds Calculation: The system shall calculate real-time quotas using the validated *Baudrier* formulas.

FR-4 Pass-and-Play: Support for local multiplayer through a single terminal interface.

0.3.2 Data Persistence

FR-5 Local Storage: Game states and statistics shall be stored in local JSON or SQLite files.

FR-6 Integrity Check: The system shall verify the checksum of local save files to prevent unauthorized manual editing of scores.

0.3.3 Required Activity Diagram

The full lifecycle of a hand, including the opening of the hidden bets window, successive betting rounds, and the final score calculation, is illustrated by the UML activity diagram below.

Activity diagram - Texas Hold'em with Recurring Hidden Bets and Strategy Score

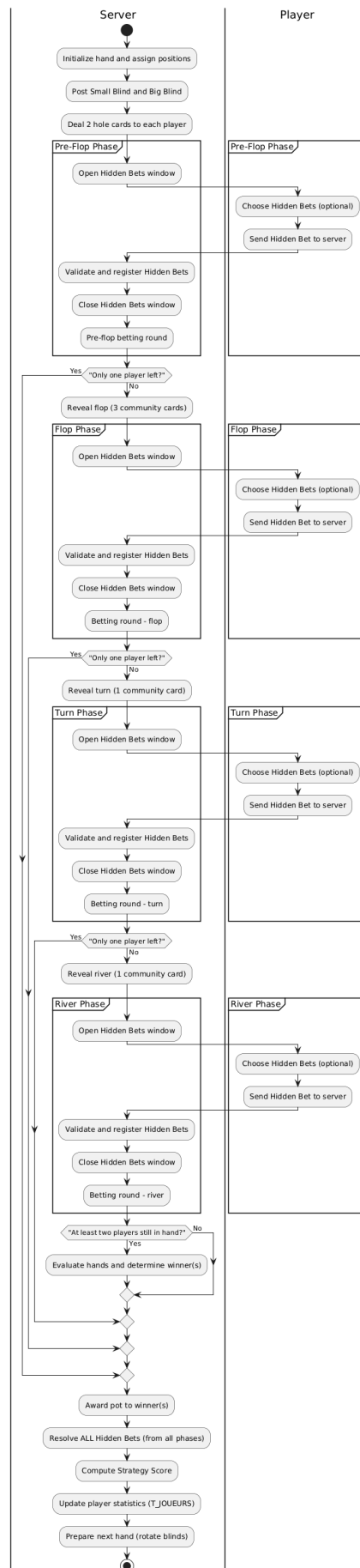


Figure 1: Texas Hold'em Hand Lifecycle with Hidden Bets

0.4 Brainstorming: Speculative "Hidden Bets" Mechanics

Version 1 – Winning Hand Strength Bet

This hidden market is called the *Winning Hand Strength Bet*, i.e. a bet on the strength category of the winning hand.

Bet timing:

- **Opening:** only during the *preflop* phase.
- **Closing / settlement:** at *showdown*.
- The bet is always placed against the *house* (or a pooled risk fund), never directly against other players.

General condition:

The player places a Hidden Bet of the form: “If the winner of the hand wins the main pot with a given hand category, I receive a payout.”

V1 typology:

In version 1, the following categories are supported:

- If the winning hand is a **Flush**: *large payout* (high multiplier).
- If the winning hand is a **Straight**: *medium payout*.
- If the winning hand is only a **Pair**: *large payout* (interesting case, since reaching showdown with just a pair is relatively rare).

All other hand categories (High Card, Two Pair, Full House, Four of a Kind, Straight Flush, etc.) do not trigger any payout in V1, unless this market is explicitly extended in a future version.

Version 2 – Advanced Hidden Bets

Name of the markets. Version 2 introduces two additional hidden markets:

- *Premium Winning Hand Bets* (long-run, from preflop to showdown),
- *Next Card Micro-Market* (short-lived, around the Turn card).

Premium Winning Hand Bets (Preflop → Showdown): These bets are placed during the preflop phase and are settled at showdown. The player does not target a specific player, but the category of the hand that will win the main pot. The following high-strength categories are supported in V2:

- **Royal Flush**
- **Straight Flush**

- **Four of a Kind**
- **Full House**

If the winning hand at showdown matches the announced category, the bet pays out according to its configured multiplier; otherwise, the stake is lost.

Next Card Micro-Market (Turn): This market is a very short-lived speculation window focused on the next community card (the Turn). Just before the Turn is dealt, the server sends the player a random prediction, for example:

- *Suit Bet:* “The next card will be a Heart.”
- *High Card Bet:* “The next card will be greater than 10.”

The client displays the question together with: “Do you want to bet on this prediction?” with buttons **Yes** / **No**.

- If the player selects **No**, no hidden bet is created and the flow ends.
- If the player selects **Yes**, the system opens the confirmation step.

The micro-market is then closed and no further action is required from the player for this Turn.

0.5 Finalizing Quantum Bluff Game Theory

0.6 Technical Specifications

0.6.1 Technology Stack

- **Language:** Java 17.
- **UI Framework:** JavaFX (MVC Pattern).
- **Build Tool:** Maven (generating a Fat JAR via Maven Shade Plugin).
- **CI/CD:** GitLab CI (automated build and unit testing).

0.6.2 Architecture Requirements

- **Multithreading:** Separate threads for the probability engine and the UI to prevent interface freezing during complex calculations.
- **Modularity:** Decoupling of the game logic (*Azra/Souheil*) from the display components.

0.7 Validation and Quality Assurance

0.7.1 Validation Protocol (M1)

The specifications have been validated through a three-step process:

1. **Technical Review:** Conducted by Mohamed to ensure architectural feasibility.
2. **QA Review:** Conducted by Linda to ensure requirements traceability.
3. **Final Approval:** Signed by Omar on February 15, 2026.

0.7.2 Non-Functional Requirements

- **Performance:** Quota calculation must return results in less than 100ms.
- **Security:** Protection of the "Hidden Bets" logic from simple decompilation.